

Dynamic Programming

It is one of the algorithmic design approach which breaks a problem into several smaller sub-problems and saves the result for future purposes so that we don't need to compute the result again. This process is called Memorization.

The subproblems are optimized to optimize the overall solution which is called Optimal Substructure Property.

The main use of dynamic programming is to solve optimization problem which means finding the best (minimum/maximum) solution for a given situation. Dynamic programming guarantees to find optimal solution if it exists.

There are 2 approaches to Dynamic Programming

- 1) Top Down Approach
- 2) Bottom Up Approach

Top Down Approach:-

It follows the memorization technique. Here memorization is equal to the sum of recursion and caching. Recursion means calling the function itself and caching means storing the intermediate results.

- easy to understand
- solves the subproblems only when required
- easy to debug.
- require more memory for recursion which degrades performance (drawback).

for a Computer Professional, there are better algorithms. One should study a standard set of important algorithms in addition to their own.

Sem V.

Design

Unit I.

Unit II

Dynamic Programming

It is one of the algorithmic design approach which breaks a problem into several smaller sub-problems and saves the result for future purposes so that we don't need to compute the result again. This process is called Memorization.

The subproblems are optimized to optimize the overall solution which is called Optimal Substructure Property.

The main use of dynamic programming is to solve optimization problem which means finding the best (minimum / maximum) solution for a given situation. Dynamic programming guarantees to find optimal solution if it exists.

There are 2 approaches to Dynamic Programming

- 1) Top Down Approach
- 2) Bottom Up Approach

Top Down Approach:-

It follows the memorization technique. Here memorization is equal to the sum of recursion and caching. Recursion means calling the function itself and caching means storing the intermediate results.

- easy to understand
- solves the subproblems only when required
- easy to debug.
- require more memory for recursion which degrades performance (drawback).

Bottom Up Approach

It uses tabulation method to implement dynamic approach. It solves the same kind of problems but it removes the recursion. In this, we solve the problems and store the results in a matrix.

Greedy Approach

It is an approach for solving a problem by selecting the best option available at the moment. It doesn't worry whether the current best result will bring the overall optimal result.

This algorithm never reverses the earlier decision even if the choice is wrong. It works in top down approach. This algorithm doesn't always produce the best result for all the problems. It's because it always goes for the local best choice to produce the global best result.

We can determine, if the algorithm can be used with any problem if the problem has following properties:

i) Greedy Choice Property

If an optimal solution to the problem can be found by choosing the best choice at each step without reconsidering the previous step once chosen then the problem can be solved using greedy approach. This is called greedy choice property.

ii) Optimal Substructure

If the optimal overall solution to the problem corresponds to the optimal solution to its subproblem then the problem can be solved using a greedy approach.

greedy approach is easy to describe.

- can perform better than other algorithms (but not in all cases)
- doesn't always produce optimal result (drawback)

Huffman Tree Construction - A Greedy Approach

A Huffman tree is a fully binary tree in which each leaf of the tree corresponds to a letter in the given alphabet.

The Huffman tree or the Huffman code is a technique of compressing data to reduce its size without losing any of its details. It was developed by David Huffman. It's also called lossless compression algorithm.

Let's understand what it is Huffman Code/Tree.

Suppose, we have 10^5 characters in a data file. The normal storage for it is 8 bits per character (ASCII). So for 10^5 characters, we need 8×10^5 bits to store them. But we want to compress the file and save it compactly. Now suppose only 6 characters appear in the file:-

Character	a	b	c	d	e	f	Total
frequency	45	13	12	16	9	5	100

Now, how can we represent the data in a compact way?

There are 2 ways:-

- i) Fixed Length Code:- In this, each character is represented by an equal no. of bits. With a fixed

length code, at least 3 bits per character are requir.

Ex:-

a = 000

b = 001

c = 010

d = 011

e = 100

f = 101

So, for a file with 10^5 characters, we need 3×10^5 bits instead of 8×10^5 bits.

ii) **Variable length Code**:- It's better than fixed length code because it gives short codes to many characters and assigns long codewords to infrequent characters.

Ex:-

a = 0

b = 101

c = 100

d = 111

e = 1101

f = 1100

$$\begin{aligned} \text{No of bits} &= \left(\underset{(a)}{45 \times 1} + \underset{(b)}{13 \times 3} + \underset{(c)}{12 \times 3} + \underset{(d)}{16 \times 3} + \underset{(e)}{9 \times 4} + \underset{(f)}{5 \times 4} \right) \times 1000 \\ &= 2.24 \times 10^5 \text{ bits} \end{aligned}$$

Thus it is an optimal code.

Now understand how can we use Greedy Algorithm for constructing a Huffman Tree / Code.

- This code works in bottom up manner
- Starts with set of $|C|$ leaves. (C is the no. of characters)
- Performs $|C|-1$ merging operation to create final tree.

understand Huffman Tree construction using greedy approach.

Consider a message M having 100 characters in total. Let's assume that the message contains 5 characters with different frequencies mentioned below:-

character	frequency
a	50
b	10
c	30
d	5
e	3
f	2

Usually, we represent (encode) characters using ASCII code which is a 7 bit character code. So, here in our example, we are assuming 100 characters and for representing these 100 characters $7 \times 100 = 700$ bits will be required.

Now second method of compression / encoding is dependent on the no. of characters. ~~Now~~ I have total 6 characters in this example, so if I want to represent them in minimum no. of bits, it would be 3 bits because $2^3 = 8$ and we have 6 characters, so minimum 3 bits are required for this. Total bits required here is $3 \times 100 = 300$ bits, which is very less as compared to 700 bits of ASCII coding.

After observing the above 2 compression / encoding methods, let's see how our Greedy Approach works. In this method, we will start by taking all the characters and their frequencies in a sorted manner.

Unit-2

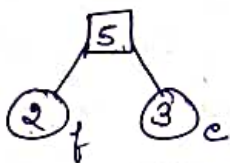
Sonal Saxena

This process use bottom up approach, so we will start from leaf nodes (terminal nodes). Let us see the sorted table of characters first:-

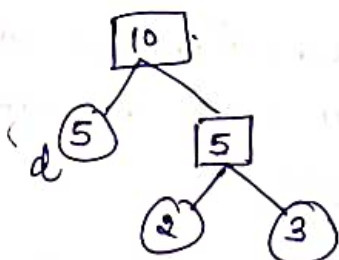
character	frequency
f	2
e	3
d	5
b	10
c	30
a	50

Here, the rules for constructing the tree includes:-
The left child should (left node) be smaller than the right child (right node) and a parent node is created by merging the left and right node (because we are using bottom up design). Let's do this on the available data.

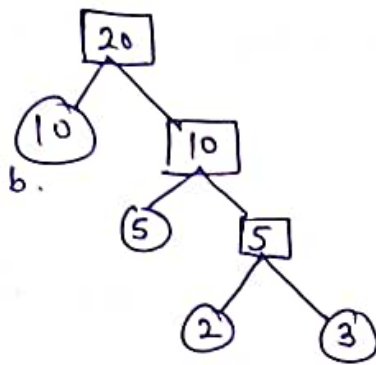
The two smallest frequencies are 2 and 3, so start making a tree with them,



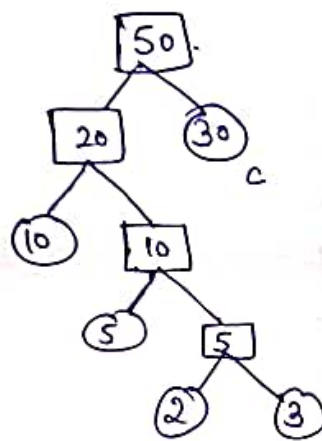
next smallest is $d = 5$. Now this 5 is equal to the parent node 5 created earlier. So we can keep this either in left or right. But then, we always have to put same value at that particular side (left/right)



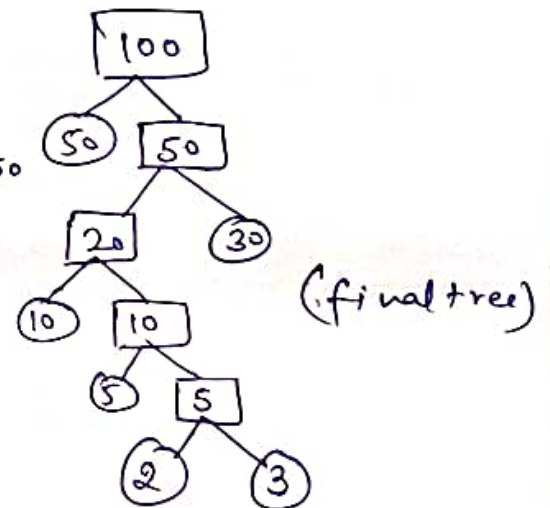
Next smallest element is $b = 10$,



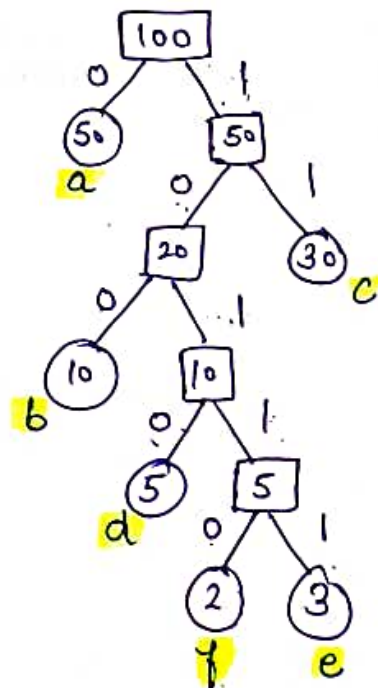
Next smallest element is $c = 30$ which is greater than 20



adding $a = 50$
 $\Rightarrow$



Now for each parent node (including root) assign '0' code to its left link and '1' code to its right link/child.



Now, we will create variable length codes ~~for each~~ ^{for each} character.

Each character,

Trace the path for each character joining 0's and 1's coming in their path:

$$a = 0 = 1 \text{ bits.}$$

$$b = 100 = 3$$

$$c = 11 = 2$$

$$d = 1010 = 4$$

$$e = 10111 = 5$$

$$f = 10110 = 5$$

← These are the trace trails of characters in the Huffman Tree.

Counting total bits :- freq. of char.

$$1 \times 50 = 50$$

$$3 \times 10 = 30$$

$$2 \times 30 = 60$$

$$4 \times 5 = 20$$

$$5 \times 3 = 15$$

$$5 \times 2 = 10$$

185 bits.

Total 185 bits are required to encode the 100 characters message using Huffman coding which is less than 700 and 300 bits. Now if you want to find average no. of bits required for each character, you can do it,

$$\frac{\text{Total bits}}{\text{Total char.}} = \frac{185}{100} = 1.85 \frac{\text{bits}}{\text{character}}$$

Knapsack Problem (Fractional) - A Greedy Approach

Problem Statement :- A thief enters into a store to steal various valuable items. To put the items, the thief has got one knapsack. This knapsack can hold a limited weight, which is a constraint. Now the greedy approach says that the thief should select the items on the basis of value of the item (profit) and weight of the item (limitation of knapsack) which also includes the concept that he can even steal fraction (some part) of the item as well.

Now, let us understand the whole process with an example:- See the table below;

items (I)	1	2	3	4	5
Weight (kg) (W)	3	3	2	5	1
Value (Profit) (P)	10	15	10	12	8

For the given set of items, find the subsets of items to be added in the knapsack weighing 10 Kgs. such that the profit is maximum and the knapsack limit is not crossed.

Find the (P_i/W_i) for each item.

item	1	2	3	4	5
Weight (W)	3	3	2	5	1
Value (P)	10	15	10	12	8
P_i/W_i	3.3	5	5	2.4	8

Sort the items in decreasing order of (P_i/W_i) ratio

Unit II

Sonal Saxena

	1	2	3	4	5
Items	5	2	3	1	4
weight	1	3	2	3	5
Pi/wi	8	5	5	3.3	2.4
Profit	8	15	10	10	12

Without exceeding the knapsack capacity, insert the items with maximum profit.

	ks wt.	wt. of item		Profit
1.)	10	- 1	= 9 kg left	8
2.)	9	- 3	= 6 kg left.	8 + 15
3.)	6	- 2	= 4 kg left	8 + 15 + 10
4.)	4	- 3	= 1 kg left	8 + 15 + 10 + 10
5.)	1	*	= fractional profit	8 + 15 + 10 + 10 + $\frac{1}{5} \times 12$

$$\Rightarrow 8 + 15 + 10 + 10 + 2.4$$

So The total profit gained is 45.4 Rs. = 45.4

$$\frac{\text{Remaining wt}}{\text{wt. of item}(x)} \times \text{profit of item}(x)$$

Longest Common Subsequence (LCS)

Problem Statement:- we need to find a common subsequence between 2 given strings S_1 and S_2 which should be longest too. The order of characters in the subsequence should be same in both the strings. For Eg:-

$S_1 = B, C, D, A, A, C, D$

$S_2 = A, C, D, B, A, C$

The common subsequences in the above 2 strings are:-

- (i) $\{B, C\}$ (ii) $\{C, D, A, C\}$ (iii) $\{D, A, C\}$ (iv) $\{A, C\}$ (v) $\{A, D\}$
 (vi) $\{A, A, C\}$ (vii) $\{C, A, C\}$.

Now among all the above subsequences, $\{C, D, A, C\}$ is the longest. But this we found manually. The same process can be done using Dynamic Programming. Let's understand the process of finding LCS using Dynamic Programming:-

Step 1:- Create a table of dimension $(n+1 * m+1)$, where 'n' and 'm' are the lengths of x and y strings respectively. In this table, first row and first column are filled with zero.

Here, $S_1 = ACADB$, $S_2 = CBDA$.
 Here, we are taking example of S_1 and S_2 again where $S_1 = x$ and $S_2 = y$. So, $n = 5$ and $m = 4$

		C	B	D	A
A					
C					
A					
D					
B					

(Table Initialization)

Design & Analysis of Algorithms

Unit I

Session V

Introduction

a Computer Professor

ical reasons to

is to know a

different areas of

able to design in

very

- study of Algorithms

ever reason for stu

developing anal

be seen as opt

are one Algorithm

enthus are a

solving a probi

part for any

them.

e important no

able program or

logic to be

can be descr

Characteristics

well defined

well defined

unambiguity

Unit II

Search Trees

(30)

Step 2:- Fill each cell of the table, using following logic:-

Step 1:-



Longest Common Subsequence (LCS)

Q19

Let A and B be two strings over an alphabet such that $A = \langle a_1, a_2, \dots, a_m \rangle$ and $B = \langle b_1, b_2, \dots, b_n \rangle$. We will say B is the subsequence of A if B can be generated by striking out some elements from the string A . This problem is designed to find the longest common subsequence of the given 2 sequences A and B .

Theorem :-

Let $A = \langle a_1, a_2, \dots, a_m \rangle$ and $B = \langle b_1, b_2, \dots, b_n \rangle$ then,

- If $a_m = b_n$, the LCS of A_m and B_n can be found by appending $a_m (= b_n)$ to LCS of A and B .
- If $a_m \neq b_n$ then it implies
 - LCS of A_{m-1} and B or
 - LCS of A and B_{n-1}

L-53

Let $c[i, j]$ be the length of an LCS of A and B , then we get

$$c[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ and/or } j=0 \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } a_i = b_j \text{ "K"} \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } a_i \neq b_j \end{cases}$$

from this we can make $c[i, j]$ table and direction arrow shows the direction.

Solving the given problem using above formula (table)

row \ col	1	2	3	4	5	6	7
m A[i]	X	Y	Z	X	T	Z	X
n B[j]	Y	T	Z	X	Y	X	

XYZY

Here $i=7$ and $j=6$. So the table will be as shown on the next page -

A
B

of the as

0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Initially set 1st row and column as 0 as given in algorithm.
 Now for $(i \leftarrow 1 \text{ to } m)$ and for $(j \leftarrow 1 \text{ to } n)$, we go on filling
 $c[i, j]$ and $d[i, j]$, horizontally
 let $i=1$ and $j=1$

if $A_i \neq B_j$
 $c[1, 1] = c[1, 0] + c[0, 1]$ is true
 $c[1, 1] \leftarrow c[0, 1]$
 $c[1, 1] \leftarrow c[0, 1]$
 $c[1, 1] \leftarrow 0$
 and $d[1, 1] \leftarrow 1$

Step 2: $i=1$ and $j=2$
 $A_i \neq B_j$
 $c[1, 2] \geq c[1, 1]$
 $c[1, 2] \leftarrow c[1, 1]$
 $c[1, 2] \leftarrow c[0, 2]$
 $c[1, 2] \leftarrow 0$
 $d[1, 2] \leftarrow 1$

Step 3: $i=1, j=3$
 $A_i \neq B_j$
 then if $c[1, 3] \leftarrow c[0, 3]$
 $c[1, 3] \leftarrow 0$
 $d[1, 3] \leftarrow 1$

Step 4: $i=1, j=4$
 $A_i \neq B_j$
 $c[1, 4] = c[0, 4] + 1$
 $c[1, 4] = c[0, 3] + 1$
 $c[1, 4] = 1$
 $d[1, 4] = 1$

Calling the
intermediate
and

(2)

calculating $c(i,j) \geq c(i,j-1)$
i.e. $c[0,5] \geq c[1,4]$
 $0 \geq 1$ which is not true

So we'll check if $c(i,j) < c(i,j-1)$

$c[1,5] \leftarrow c[1,4]$
i.e. $c[1,5] \leftarrow c[1,4]$
 $c[1,5] \leftarrow 1$
 $d[1,5] \leftarrow "a"$

Step 4 $i=1, j=6, A[i] = B[j]$

then
 $c[1,6] \leftarrow c[1,5] + 1$
 $c[1,6] \leftarrow c[0,5] + 1$
 $c[1,6] \leftarrow 1$
 $d[1,6] \leftarrow "a"$

Step 5 $i=2, j=1$

$A[i] \neq B[j]$
 $c[2,1] \leftarrow c[2,0] + 1$
 $c[2,1] \leftarrow c[1,0] + 1$
 $c[2,1] \leftarrow 0 + 1$
 $c[2,1] \leftarrow 1$
 $d[2,1] \leftarrow "a"$

Step 6 $i=2, j=2, A[i] \neq B[j]$

checking if $c[2,1] \geq c[1,2]$
 $c[2,1] \geq c[1,2]$
 $1 \geq 2$ not true, so
 $c[2,2] \leftarrow c[2,1]$
 $c[2,2] \leftarrow c[1,2]$
 $c[2,2] \leftarrow 1$
 $d[2,2] \leftarrow "a"$

C-159

likewise we'll find out all $c[i,j]$ and $d[i,j]$ so the complete formed table is shown below:

$i=3$
 $j=6$

$c[2,6] \geq c[3,5]$
 $2 \geq 2$
 $c[2,6]$

length matrix

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	↑0	↑0	↑0	↖1	←1	←1
2	0	↖1	←1	←1	↑1	↖2	←2
3	0	↑1	↑1	↖2	↖2	↑2	↖3
4	0	↖1	↑1	↑2	↑2	↖3	↖3
5	0	↑1	↖2	↑2	↑2	↖3	↑3
6	0	↑1	↑2	↑2	↖3	↑3	↖4
7	0	↖1	↑2	↑2	↑3	↖4	↑4

(57)
22

c, d

length
d = 7
c = 6
j = [n, w]

Now, we can construct a LCS using $c[i, j]$ and $d[i, j]$

To derive the LCS we have to make use of d-table. The algo for this is as shown below.

Algo display-LCS($d, A[1..n], c[i, j]$)

if ($i=0$ || $j=0$) then

return

if ($d[i, j] = \text{"↖"}$) then

display-LCS($d, A, c[i-1, j-1]$)

print(a_i)

else if ($d[i, j] = \text{"↑"}$) then

display-LCS($d, A, c[i, j-1]$)

else

display-LCS($d, A, c[i, j-1]$)

}

$d[7, 6]$

6, 6 → a₆ → X

5, 5

4, 5 → a₄ → Y

3, 4

3, 3 → a₃ → Z

0, 2, 2

2, 1 → a₂ → Y

1, 0

7, 6

6, 6

5, 5

4, 5

3, 4

d, A, (6, 6) YZ YX

96.

94

Initially, a call is given as display-LCS($d, A[1..7], c$)

As $d[7, 6] = \text{"↑"}$, a recursive call is made to ($d, A[1..7], c[6, 6]$)

Here $d[6, 6] = \text{"↖"}$, then a call is made to ($d, A[1..6], c[5, 5]$)

Several
are built
in. This
are a
is call
of dyn
blem
num)
exist
bad
bad
proo

call is made to $dp(d, i, j)$. $d[4,5] = \nwarrow$ print $a_i = y$
is "Y", then again

complete steps are written below:-

23

- ① $d[7,6] = \uparrow$
→ $d[6,6] (d, A, i-1, j)$
- ② $d[6,6] = \nwarrow$
→ $d[5,5] (d, A, i-1, j-1)$
→ print $a_6 = X$
- ③ $d[5,5] = \uparrow$
→ $d[4,5] (d, A, i-1, j)$
- ④ $d[4,5] = \nwarrow$
→ $d[3,4] (d, A, i-1, j-1)$
print $a_4 = Y$
- ⑤ $d[3,4] = \nwarrow$
→ $d[3,3] (d, A, i, j-1)$
- ⑥ $d[3,3] = \nwarrow$
→ $d[2,2] (d, A, i-1, j-1)$
→ print $a_3 = Z$
- ⑦ $d[2,2] = \nwarrow$
→ $d[2,1] (d, A, i, j-1)$
- ⑧ $d[2,1] = \nwarrow$
→ $d[1,0] (d, A, i-1, j-1)$
return as $j=0$ here
print $a_2 = Y$

So Now, as A and B were -

A { X [✓] Y [✓] Z [✓] Y T X Y [✓] }
B { Y T Z X Y X }
← starting from here
← starting from here

or the LCS is YZYX

Algorithm: LCS-Length-Table-Formulation ()

$m := \text{length}(X)$

$n := \text{length}(Y)$

for $i = 1$ to m do

$C[i, 0] := 0$

for $j = 1$ to n do

$C[0, j] := 0$

for $i = 1$ to m do

for $j = 1$ to n do

if $x_i = y_j$

$C[i, j] := C[i - 1, j - 1] + 1$

$B[i, j] := 'D'$

else

if $C[i - 1, j] \geq C[i, j - 1]$

$C[i, j] := C[i - 1, j] + 1$

$B[i, j] := 'U'$

else

$C[i, j] := C[i, j - 1] + 1$

$B[i, j] := 'L'$

return C and B

Algorithm: Print-LCS (B, X, i, j)

if $i = 0$ and $j = 0$

return

if $B[i, j] = 'D'$

Print-LCS($B, X, i-1, j-1$)

Print(x_i)

else if $B[i, j] = 'U'$


```

Algorithm: Print-LCS (B, X, i, j)
if i = 0 and j = 0
    return
if B[i, j] = 'D'
    Print-LCS(B, X, i-1, j-1)
    Print(xi)
else if B[i, j] = 'U'
    Print-LCS(B, X, i-1, j)
else
    Print-LCS(B, X, i, j-1)

```

(25)

X = A B C D A B
Y = E D C A B A

This algorithm will print the longest common subsequence of X and Y.

Analysis

ABRAC. > ABA.
YABBAD.

To populate the table, the outer for loop iterates m times and the inner for loop iterates n times. Hence, the complexity of the algorithm is $O(m, n)$, where m and n are the length of two strings.

Example

In this example, we have two strings $X = BACDB$ and $Y = BDCB$ to find the longest common subsequence.

Following the algorithm LCS-Length-Table-Formulation (as stated above), we have calculated

Initially, $\text{table}[i, j] = 0$ and $\text{table}[i, 0] = 0$. There are 0 to w columns and 0 to w columns in the table.

$\text{col}(j)$ $\text{knapsack}(w)$ $w+1$

	0	1	2	3	4	5
0	0					
1	0					
2	0					
3	0					
4	0					
5	0					

$i(2 \times w)$
Items
 $i+1$

(11)

Item	wt	pt
1	2	3
2	3	4
3	4	5
4	5	6

Now we'll fill up the table either row by row or column by column. Let us start row by row.

$$\text{table}[i, j] = \begin{cases} \max\{\text{table}[i-1, j], v_i + \text{table}[i-1, j-w_i]\} & \text{when } j \geq w_i \\ \text{table}[i-1, j] & \text{if } j < w_i \end{cases}$$

C-53

table[1, 1] with $i=1, j=1, w_i=2, v_i=3$
As $j < w_i$ i.e. $1 < 2$, so

$$\text{table}[1, 1] = \text{table}[1-1, 1] = \text{table}[0, 1] = 0$$

$$\text{So } \text{table}[1, 1] = 0$$

table[1, 2] with $i=1, j=2, w_i=2, v_i=3$

As $j \geq w_i$ we'll obtain $\text{table}[1, 2]$ as

$$\begin{aligned} \text{table}[1, 2] &= \max\{\text{table}[0, 2], 3 + \text{table}[0, 0]\} \\ &= \max\{0, 3 + 0\} \end{aligned}$$

$$\text{table}[1, 2] = 3$$

table[1, 3] with $i=1, j=3, w_i=2, v_i=3$

As $j \geq w_i$ we'll obtain $\text{table}[1, 3]$ as

$$\text{table}[1, 3] = \max\{\text{table}[0, 3], v_i + \text{table}[0, 1]\}$$

$$= \max\{0, 3 + 0\}$$

$$\text{table}[1, 3] = 3$$

table [1,4] with $i=1, j=4, w_i=2, v_i=3$

As $j \geq w_i$, we'll obtain table [1,4] as

$$\text{table}[1,4] = \max\{\text{table}[0,4], 3 + \text{table}[0,2]\}$$

$$= \max\{0, 3\}$$

$$\text{table}[1,4] = \boxed{3}$$

(9)

(12)

table [1,5] with $i=1, j=5, w_i=2$ and $v_i=3$

As $j \geq w_i$, we'll obtain table [1,5] as

$$\text{table}[1,5] = \max\{\text{table}[0,5], 3 + \text{table}[0,3]\}$$

$$= \max\{0, 3 + 0\}$$

$$\text{table}[1,5] = \boxed{3}$$

Now the table we have is —

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0					
3	0					
4	0					
5	0					

table [2,1], $w_i=3$ and $v_i=4$

As $j < w_i$,

$$\text{table}[2,1] = \text{table}[i-1, j]$$

$$= \text{table}[1,0]$$

$$\text{table}[2,1] = \boxed{0}$$

table [2,2]

As $j < w_i$, $w_i=3, v_i=4$

$$\text{table}[2,2] = \text{table}[i-1, j]$$

$$= \text{table}[1,2]$$

$$\text{table}[2,2] = \boxed{3}$$

table [2,3]

As $j \geq w_i$

$$\text{table}[2,3] = \max\{\text{table}[i-1, j], v_i + \text{table}[i-1, j-w_i]\}$$

$$= \max\{\text{table}[1,3], 4 + \text{table}[1,0]\}$$

$$= \max\{3, 4 + 0\}$$

$$\text{table}[2,3] = \boxed{4}$$

$i=2, j=4, w_i=3$ and $v_i=4$

$$\begin{aligned} \text{table}[2,4] &= \max\{\text{table}[1,4], 4 + \text{table}[1,3]\} \\ &= \max\{3, 4 + 0\} \\ &= \max\{3, 4\} \\ \text{table}[2,4] &= \boxed{4} \end{aligned}$$

table[2,5] $i=2, j=5, w_i=3$ and $v_i=4$
 $j \geq w_i$

$$\begin{aligned} \text{table}[2,5] &= \max\{\text{table}[1,5], 4 + \text{table}[1,2]\} \\ &= \max\{3, 4 + 3\} \\ &= \max\{3, 7\} \\ \text{table}[2,5] &= \boxed{7} \end{aligned}$$

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0					
4	0					
5	0					

table[3,1] $i=3, j=1, w_i=4$, and $v_i=5$
 $j < w_i$

$$\begin{aligned} \text{So, table}[3,1] &= \text{table}[i-1, j] \\ &= \text{table}[2,1] \\ \text{table}[3,1] &= \boxed{0} \end{aligned}$$

table[3,2] $i=3, j=2, w_i=4, v_i=5$
 $j < w_i$

$$\begin{aligned} \text{table}[3,2] &= \text{table}[2,2] \\ \text{table}[3,2] &= \boxed{3} \end{aligned}$$

table[3,3] $i=3, j=3, w_i=4, v_i=5$
 $j < w_i$

$$\begin{aligned} \text{table}[3,3] &= \text{table}[2,3] \\ \text{table}[3,3] &= \boxed{4} \end{aligned}$$

table [3,4] with $i=3, j=4, w_i=4, v_i=5$
 $j \leq w_i$

$$\begin{aligned} \text{table}[3,4] &= \max \{ \text{table}[i-1, j], v_i + \text{table}[i-1, j-w_i] \} \\ &= \max \{ \text{table}[2,4], 5 + \text{table}[2,0] \} \\ &= \max \{ 4, 5+0 \} \\ \text{table}[3,4] &= \boxed{5} \end{aligned}$$

(5)

(14)

table [3,5] with $i=3, j=5, w_i=4, v_i=5$
 $j \geq w_i$

$$\begin{aligned} \text{table}[3,5] &= \max \{ \text{table}[2,5], 5 + \text{table}[2,1] \} \\ &= \max \{ 7, 5+0 \} \\ \text{table}[3,5] &= \boxed{7} \end{aligned}$$

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3			
5						

table [4,1] $i=4, j=1, w_i=5, v_i=6$
 $j < w_i$

$$\begin{aligned} \text{table}[4,1] &= \text{table}[i-1, j] \\ &= \text{table}[3,1] \\ \text{table}[4,1] &= \boxed{0} \end{aligned}$$

table [4,2] $i=4, j=2, w_i=5, v_i=6$
 $j < w_i$

$$\begin{aligned} &= \text{table}[3,2] \\ \text{table}[4,2] &= \boxed{3} \end{aligned}$$

table [4,3]

$i=4, j=3, w_i=5, v_i=6$
 $j < w_i$

$$\begin{aligned} \text{table}[4,3] &= \text{table}[3,3] \\ \text{table}[4,3] &= \boxed{4} \end{aligned}$$

size table $[4,4] = 5$

table $[4,5] = 7$

Now the table is :-

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

→ This is the total value of selected items. (max profit)

$$S = W_0$$

$$C = W_0$$

15

How to find actual Knapsack items? 2 (Algo)

As we know that table $[n, W]$ is the total value of selected items.

Now finding the items which are included -

Let $i = n$ and $K = W$ (while $i > 0$ and $K > 0$)

So $i = 4$, $K = 5$

If (table $[i, K] \neq$ table $[i-1, K]$) then

mark i^{th} item as in Knapsack. ✓

$i = i-1$ and $K = K - w_i$ // selection of i^{th} item

else

$i = i-1$ // don't select i^{th} item.

We have $i = 4$, $K = 5$.

As table $[4, 5] =$ table $[3, 5]$

Set $i = 4-1$
 $= 3$

4th item is not selected

table $[3, 5] =$ table $[2, 5]$

don't select 3rd item

and Set $i = 3-1$

$i = 2$

3rd item is not selected

~~X~~ ~~X~~ 2^{i-1}

$i = 3$

Algo -

$$5 - 3 = 2$$

$$C[1, 2] = C[0, 2]$$

table[2,5] $\neq$ table[1,5]

Select 2nd item

Set $i = 2 - 1 = 1$ and $K = K - w_i = 5 - 3 = 2$

table[1,2] $\neq$ table[0,2]

Select 1st item

Set $i = 1 - 1 = 0$, and $K = K - w_i = 2 - 2 = 0$

Thus we have selected item 1 and 2 for the Knapsack.
This solution can also be represented by a vector (1, 1, 0, 0)

Algorithm: -

Dynamic-Knap($n, W, w[], v[]$)

n is the total no. of items, W is the Knapsack size, $w[]$ and $v[]$ stores weights and values of the items respectively.
This algo returns the total value of selected items for the Knapsack.

for ($i \leftarrow 0$ to n) do

{

for ($j \leftarrow 0$ to W) do

{

table[$i, 0$] = 0 // table initialization

table[$0, j$] = 0

}

}

for ($i \leftarrow 0$ to n) do

{

for ($j \leftarrow 0$ to W) do

{

if ($j < w[i]$) then

table[i, j] $\leftarrow$ table[$i-1, j$]

else if ($j \geq w[i]$) then

table [2,5] $\neq$ table [1,5]

Select 2nd item,

Set $i = 2 - 1 = 1$ and $K = K - w_i = 5 - 3 = 2$

(16)

table [1,2] $\neq$ table [0,2]

Select 1st item

Set $i = 1 - 1 = 0$, and $K = K - w_i = 2 - 2 = 0$

Thus we have selected item 1 and 2 for the knapsack.
This solution can also be represented by a vector (1, 1, 0, 0)

Algorithm: -

Dynamic-Knap ($n, W, w[], v[]$)

n is the total no. of items, W is the knapsack size, $w[]$ and $v[]$ stores weights and values of the items respectively.
This algo returns the total value of selected items for the knapsack.

for ($i \leftarrow 0$ to n) do

{

for ($j \leftarrow 0$ to W) do

{

table [$i, 0$] = 0 // table initialization

table [$0, j$] = 0

}

}

for ($i \leftarrow 0$ to n) do

{

for ($j \leftarrow 0$ to W) do

{

if ($j < w[i]$) then

table [i, j] $\leftarrow$ table [$i-1, j$]

else if ($j \geq w[i]$) then

$$[i,j] \leftarrow \max \{ \text{table}[i-1, j], v_i + \text{table}[i-1, j-w_i] \}$$

}
 }
 return table[n,w]

Time complexity $\rightarrow O(nw)$

(17)

2010
2

1050

B = ACT ✓

Example: — $\langle x, z, y, x \rangle$ is a subsequence of

$\langle x, y, x, z, y, z, y, x, z \rangle$

(18)

Computing length of a LCS.

Algorithm for computing $LCS(A, B)$

This algo builds $c[i, j]$ and $d[i, j]$ table

$m \leftarrow \text{length}(A)$

$n \leftarrow \text{length}(B)$

for ($i \leftarrow 1$ to m) do

$c[i, 0] \leftarrow 0$

for ($j \leftarrow 1$ to n) do

$c[0, j] \leftarrow 0$

for ($i \leftarrow 1$ to m) do

{ for ($j \leftarrow 1$ to n) do

{ if ($a_i = b_j$) then

{

$c[i, j] \leftarrow c[i-1, j-1] + 1$

{ $d[i, j] \leftarrow \nwarrow$ // putting direction

} else if ($c[i-1, j] \geq c[i, j-1]$) then

{

$c[i, j] \leftarrow c[i-1, j]$

$d[i, j] \leftarrow \uparrow$

}

else

{ $c[i, j] \leftarrow c[i, j-1]$

$d[i, j] \leftarrow \leftarrow$

}

}

} return c and d

}

BAND
TAND

$a_i \neq b_j$

$c(0, 1)$

$c(i-1, j)$
 $c(0, 1)$

WV

$$B = \overline{A C T} \checkmark$$

Example: $\langle x, y, z, x \rangle$ is a subsequence of
 $\langle x, y, z, z, y, z, y, x, z \rangle$

Computing length of a LCS

(18)

max len for computing LCS (A, B)
 since algo builds $c[i, j]$ and $d[i, j]$ table

7 max length (A)
 8 max length (B)

```
for (i ← 1 to m) do
  c[i, 0] ← 0
  for (j ← 1 to n) do
    c[0, j] ← 0
  for (i ← 1 to m) do
```

```
  {
    for (j ← 1 to n) do
      {
        if (ai = bj) then
```

```
        c[i, j] ← c[i-1, j-1] + 1
```

```
        {
          d[i, j] ← "↖" // putting direction
        }
```

```
      elseif (c[i-1, j] > c[i, j-1]) then
```

```
      {
        c[i, j] ← c[i-1, j]
        d[i, j] ← "↑"
```

```
      }
```

```
    else
      {
        c[i, j] ← c[i, j-1]
```

```
        d[i, j] ← "←"
```

```
      }
```

```
    }
  }
  return c and d
```

B A N D
 I A N D

a_i = b_j

c[i, j]

c[i-1, j]
 c[i, j-1]

the